

1、问题说明

1.1 问题描述

在 `Langchain1.2.12` 中，调用深度求索官方模型时，如果启用了思考模式，且触发工具调用，则报错如下

```
1 | Traceback...
2 | BadRequestError: Error code: 400 - {'error': {'message': 'The
   | `reasoning_content` in the thinking mode must be passed back to the API.',
   | 'type': 'invalid_request_error', 'param': None, 'code':
   | 'invalid_request_error'}}
```

错误信息告诉我们：在 `thinking mode`（思考模式）下，必须将 `reasoning_content` 传回 API。

1.2 背景知识

目前深度求索官方提供两种模型

- deepseek-v4-flash
- deepseek-v4-pro

它们都支持思考模式和非思考模式，默认情况下为思考模式，[参考官方文档](#)

模型细节

模型		deepseek-v4-flash ⁽¹⁾	deepseek-v4-pro
BASE URL (OpenAI 格式)		https://api.deepseek.com	
BASE URL (Anthropic 格式)		https://api.deepseek.com/anthropic	
模型版本		DeepSeek-V4-Flash	DeepSeek-V4-Pro
思考模式		支持非思考与思考模式（默认） 切换方式详见思考模式	
上下文长度		1M	
输出长度		最大 384K	
功能	Json Output	支持	支持
	Tool Calls	支持	支持
	对话前缀续写 (Beta)	支持	支持
	FIM 补全 (Beta)	仅非思考模式支持	仅非思考模式支持
价格	百万tokens输入（缓存命中） ⁽²⁾	0.02元	0.025元 (2.5折 ⁽³⁾) 0.1元
	百万tokens输入（缓存未命中）	1元	3元 (2.5折 ⁽³⁾) 12元

思考模式可以通过请求体中的特定字段控制，[参考官方文档](#)

思考模式开关与思考强度控制

	控制参数 (OpenAI 格式)	控制参数 (Anthropic 格式)
思考模式开关 ⁽¹⁾	{"thinking": {"type": "enabled/disabled"}}	
思考强度控制 ⁽²⁾⁽³⁾	{"reasoning_effort": "high/max"}	{"output_config": {"effort": "high/max"}}

1.3 问题复现

```
1 from langchain_deepseek import ChatDeepSeek
2 from langchain.messages import HumanMessage
3 from langchain.tools import tool
4 from dotenv import load_dotenv
5 load_dotenv(override=True)
6
7 @tool(parse_docstring=True)
8 def get_weather(city: str):
```

```

9         """
10         获取指定城市天气
11
12         Args:
13             city: 城市名称
14         """
15         return f"{city}今天天气不错"
16
17     model = ChatDeepSeek(
18         model="deepseek-v4-flash",
19         extra_body={
20             "thinking": {
21                 "type": "enabled"
22             }
23         }
24     )
25     model_with_tools = model.bind_tools([get_weather])
26     messages = [HumanMessage("今天北京天气如何? ")]
27
28     response1 = model_with_tools.invoke(messages)
29     messages.append(response1)
30
31     for tool_call in response1.tool_calls:
32         if tool_call["name"] == "get_weather":
33             tool_msg = get_weather.invoke(tool_call)
34             messages.append(tool_msg)
35
36     response = model_with_tools.invoke(messages)
37     messages.append(response)
38
39     for msg in messages:
40         print("=" * 80)
41         print(msg)

```

运行报错如下

```

1 Traceback...
2 BadRequestError: Error code: 400 - {'error': {'message': 'The
`reasoning_content` in the thinking mode must be passed back to the API.',
'type': 'invalid_request_error', 'param': None, 'code':
'invalid_request_error'}}

```

2、根因剖析

2.1 Deepseek官方返回的响应格式

将以下命令的 **<TOKEN>** 替换为Deepseek官方提供的 **API_KEY**，然后粘贴到Linux命令行，或者用PostMan等工具，向Deepseek官方发送原始请求

```

1 curl -L -X POST 'https://api.deepseek.com/chat/completions' \
2 -H 'Content-Type: application/json' \
3 -H 'Accept: application/json' \
4 -H 'Authorization: Bearer <TOKEN>' \
5 --data-raw '{
6     "messages": [
7         {

```

```

8      "content": "你好",
9      "role": "user"
10   }
11 ],
12 "model": "deepseek-v4-flash",
13 "thinking": {
14     "type": "enabled"
15 }
16 }'

```

响应如下所示

```

1  {
2      "id": "00b0013c-5f26-4c80-8e82-1fd130aedc5f",
3      "object": "chat.completion",
4      "created": 1778061401,
5      "model": "deepseek-v4-flash",
6      "choices": [
7          {
8              "index": 0,
9              "message": {
10                 "role": "assistant",
11                 "content": "你好呀！👋 很高兴见到你！我是DeepSeek，随时准备为你提供
帮助。无论是回答问题、聊天讨论，还是协助你完成任务，我都会尽力满足你的需求。有什么我可以帮你的
吗？😊",
12                 "reasoning_content": "嗯，用户发来一句简单的问候“你好”，这是一个非
常基础的社交开场。用户可能是初次接触，想打个招呼，或者测试我是否在线。深层需求是希望得到友好的
回应，开启对话。\\n\\n我应该用同样友好、热情的语气回应，自我介绍并表达乐于助人的态度。可以
用开放性的提问，邀请用户提出具体需求，比如解释概念、聊天或者回答问题。这样既保持了亲和力，又
提供了清晰的互动方向。\\n\\n想到了直接回复“你好呀！”，加上一个微笑表情，然后简单介绍自己，最
后用几个具体选项引导用户说出意图。"
13             },
14             "logprobs": null,
15             "finish_reason": "stop"
16         }
17     ],
18     "usage": {
19         "prompt_tokens": 5,
20         "completion_tokens": 166,
21         "total_tokens": 171,
22         "prompt_tokens_details": {
23             "cached_tokens": 0
24         },
25         "completion_tokens_details": {
26             "reasoning_tokens": 119
27         },
28         "prompt_cache_hit_tokens": 0,
29         "prompt_cache_miss_tokens": 5
30     },
31     "system_fingerprint": "fp_058df29938_prod0820_fp8_kvcache_20260402"
32 }

```

如上所示，Deepseek官方返回的 **HTTP响应** 中，**思考内容**记录在 **消息对象** 的一级字段 **reasoning_content** 中。

2.2 构造消息列表

为了排除 **Langchain** 的干扰，我们构造工具调用的消息列表，直接发送请求，观察现象。

2.2.1 调用请求包含完整历史消息

```
1 curl -L -X POST 'https://api.deepseek.com/chat/completions' \  
2 -H 'Content-Type: application/json' \  
3 -H 'Accept: application/json' \  
4 -H 'Authorization: Bearer <TOKEN>' \  
5 --data-raw '{  
6   "messages": [  
7     {  
8       "content": "今天北京天气如何？",  
9       "role": "user"  
10    },  
11    {  
12      "content": "",  
13      "reasoning_content": "用户想知道今天北京的天气。我可以使用 get_weather 工具  
14      来获取北京今天的天气信息。",  
15      "role": "assistant",  
16      "tool_calls": [  
17        {  
18          "function": {  
19            "arguments": "{\"city\": \"北京\"}",  
20            "name": "get_weather"  
21          },  
22          "id": "call_00_e82qYXWR9mYwcdwNCDo1725",  
23          "type": "function"  
24        }  
25      ]  
26    },  
27    {  
28      "content": "北京今天天气不错",  
29      "role": "tool",  
30      "tool_call_id": "call_00_e82qYXWR9mYwcdwNCDo1725"  
31    }  
32  ],  
33   "model": "deepseek-v4-flash",  
34   "thinking": {  
35     "type": "enabled"  
36   }  
37 }'
```

输出如下

```
1 {  
2   "id": "1228d108-f30f-4174-9bdc-71561f422964",  
3   "object": "chat.completion",  
4   "created": 1778124129,  
5   "model": "deepseek-v4-flash",  
6   "choices": [  
7     {  
8       "index": 0,  
9       "message": {  
10        "role": "assistant",
```

```

11         "content": "北京今天天气不错。",
12         "reasoning_content": "根据天气查询结果，北京今天的天气情况是“不
    错”。我可以直接告诉用户这个结果，并简单说明一下。"
13     },
14     "logprobs": null,
15     "finish_reason": "stop"
16 }
17 ],
18 "usage": {
19     "prompt_tokens": 90,
20     "completion_tokens": 31,
21     "total_tokens": 121,
22     "prompt_tokens_details": {
23         "cached_tokens": 0
24     },
25     "completion_tokens_details": {
26         "reasoning_tokens": 25
27     },
28     "prompt_cache_hit_tokens": 0,
29     "prompt_cache_miss_tokens": 90
30 },
31 "system_fingerprint": "fp_058df29938_prod0820_fp8_kvcache_20260402"
32 }

```

此时响应正常

2.2.2 去掉工具调用请求中的reasoning_content

命令如下

```

1  curl -L -X POST 'https://api.deepseek.com/chat/completions' \
2  -H 'Content-Type: application/json' \
3  -H 'Accept: application/json' \
4  -H 'Authorization: Bearer <TOKEN>' \
5  --data-raw '{
6      "messages": [
7          {
8              "content": "今天北京天气如何？",
9              "role": "user"
10         },
11         {
12             "content": "",
13             "role": "assistant",
14             "tool_calls": [
15                 {
16                     "function": {
17                         "arguments": "{\"city\": \"北京\"}",
18                         "name": "get_weather"
19                     },
20                     "id": "call_00_e82qQYXWR9mYwcdwNCDo1725",
21                     "type": "function"
22                 }
23             ]
24         },
25         {
26             "content": "北京今天天气不错",
27             "role": "tool",

```

```

28         "tool_call_id": "call_00_e82qYXWR9mYwcdwNCDo1725"
29     }
30 ],
31     "model": "deepseek-v4-flash",
32     "thinking": {
33         "type": "enabled"
34     }
35 }'

```

报错如下

```

1  {
2      "error": {
3          "message": "The `reasoning_content` in the thinking mode must be
passed back to the API.",
4          "type": "invalid_request_error",
5          "param": null,
6          "code": "invalid_request_error"
7      }
8  }

```

报错信息和上文一致，初步推断问题出现的原因是请求体中的历史 `AIMessage` 缺少 `reasoning_content`

2.3 Langchain的消息封装

为了搞清楚问题触发的原因，需要了解Langchain底层的消息封装

2.3.1 AIMessage对象结构

执行以下代码

```

1  from langchain_deepseek import ChatDeepSeek
2
3  model = ChatDeepSeek(
4      model="deepseek-v4-flash",
5      extra_body={
6          "thinking": {
7              "type": "enabled"
8          }
9      }
10 )
11
12 response = model.invoke("你好")
13 print(response)

```

输出如下

```

1 content='你好！很高兴见到你😊\n\n我是DeepSeek，由深度求索公司创造的AI助手。无论你想聊什么、问什么，或者需要任何帮助，我都非常乐意为你效劳！\n\n今天有什么我可以帮你的吗？无论是解答问题、提供建议，还是单纯聊聊天，我都很开心能陪伴你～' additional_kwargs={'refusal': None, 'reasoning_content': '好的，用户发来一句简单的问候“你好”。这是一个非常基础的社交开场，没有具体问题或指令。用户可能是在测试我是否在线，或者准备开始一段对话。深层需求可能是建立联系，开启交流。\\n\\n我需要给出一个友好、热情且开放的回应，以鼓励用户继续表达。回复应该包括问候、自我介绍、表达帮助意愿，并用开放性问题引导进一步互动。想到了用“很高兴见到你”来建立亲切感，说明我的身份是DeepSeek助手，强调免费、功能范围（聊天、查询、分析），最后用“请问有什么我可以帮你”明确邀请用户提出需求。\\n\\n嗯，这样既回应了问候，又为后续对话铺平了道路。'} response_metadata={'token_usage': {'completion_tokens': 214, 'prompt_tokens': 5, 'total_tokens': 219, 'completion_tokens_details': {'accepted_prediction_tokens': None, 'audio_tokens': None, 'reasoning_tokens': 145, 'rejected_prediction_tokens': None}, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 5}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'id': '779d6c5e-8f2d-4875-b153-6b7e33f01e97', 'finish_reason': 'stop', 'logprobs': None} id='lc_run--019dfcba-ec41-71a0-9813-f35534addf29-0' tool_calls=[] invalid_tool_calls=[] usage_metadata={'input_tokens': 5, 'output_tokens': 214, 'total_tokens': 219, 'input_token_details': {'cache_read': 0}, 'output_token_details': {'reasoning': 145}}

```

Langchain 将响应封装在 **AIMessage** 实例中，思考内容 **reasoning_content** 存储在后者的 **additional_kwargs** 字段下。

2.3.2 Langchain封装的请求体payload格式

① 相关源码

1. _convert_message_to_dict()

源码位于 **langchain_openai.chat_models.base** 的 **_convert_message_to_dict()** 函数中，如下

```

1 def _convert_message_to_dict(
2     message: BaseMessage,
3     api: Literal["chat/completions", "responses"] = "chat/completions",
4 ) -> dict:
5     """Convert a LangChain message to dictionary format expected by
6     OpenAI."""
7     message_dict: dict[str, Any] = {
8         "content": _format_message_content(message.content, api=api,
9         role=message.type)
10    }
11    if (name := message.name or message.additional_kwargs.get("name")) is
12    not None:
13        message_dict["name"] = name
14
15    # populate role and additional message data
16    if isinstance(message, ChatMessage):
17        message_dict["role"] = message.role
18    elif isinstance(message, HumanMessage):
19        message_dict["role"] = "user"
20    elif isinstance(message, AIMessage):
21        message_dict["role"] = "assistant"
22        if message.tool_calls or message.invalid_tool_calls:
23            message_dict["tool_calls"] = [

```



```

21         _lc_tool_call_to_openai_tool_call(tc) for tc in
message.tool_calls
22     ] + [
23         _lc_invalid_tool_call_to_openai_tool_call(tc)
24         for tc in message.invalid_tool_calls
25     ]
26     elif "tool_calls" in message.additional_kwargs:
27         message_dict["tool_calls"] =
message.additional_kwargs["tool_calls"]
28         tool_call_supported_props = {"id", "type", "function"}
29         message_dict["tool_calls"] = [
30             {k: v for k, v in tool_call.items() if k in
tool_call_supported_props}
31             for tool_call in message_dict["tool_calls"]
32         ]
33     elif "function_call" in message.additional_kwargs:
34         # OpenAI raises 400 if both function_call and tool_calls are
present in the
35         # same message.
36         message_dict["function_call"] =
message.additional_kwargs["function_call"]
37     else:
38         pass
39     # If tool calls present, content null value should be None not empty
string.
40     if "function_call" in message_dict or "tool_calls" in message_dict:
41         message_dict["content"] = message_dict["content"] or None
42
43     audio: dict[str, Any] | None = None
44     for block in message.content:
45         if (
46             isinstance(block, dict)
47             and block.get("type") == "audio"
48             and (id_ := block.get("id"))
49             and api != "responses"
50         ):
51             # openai doesn't support passing the data back - only the id
52             # https://platform.openai.com/docs/guides/audio/multi-turn-
conversations
53             audio = {"id": id_}
54             if not audio and "audio" in message.additional_kwargs:
55                 raw_audio = message.additional_kwargs["audio"]
56                 audio = (
57                     {"id": message.additional_kwargs["audio"]["id"]}
58                     if "id" in raw_audio
59                     else raw_audio
60                 )
61             if audio:
62                 message_dict["audio"] = audio
63         elif isinstance(message, SystemMessage):
64             message_dict["role"] = message.additional_kwargs.get(
65                 "__openai_role__", "system"
66             )
67         elif isinstance(message, FunctionMessage):
68             message_dict["role"] = "function"
69         elif isinstance(message, ToolMessage):
70             message_dict["role"] = "tool"
71             message_dict["tool_call_id"] = message.tool_call_id

```

```

72     message_dict["content"] = _sanitize_chat_completions_content(
73         message_dict["content"]
74     )
75     supported_props = {"content", "role", "tool_call_id"}
76     message_dict = {k: v for k, v in message_dict.items() if k in
supported_props}
77     else:
78         msg = f"Got unknown type {message}"
79         raise TypeError(msg)
80     return message_dict

```

消息被转换为 `message_dict`，这一过程并没有提取 `reasoning_content`。

2. `_convert_from_v1_to_chat_completions()`

源码位于 `langchain_openai.chat_models.compat` 的 `_convert_from_v1_to_chat_completions()` 函数中，如下

```

1  def _convert_from_v1_to_chat_completions(message: AIMessage) -> AIMessage:
2      """Convert a v1 message to the Chat Completions format."""
3      if isinstance(message.content, list):
4          new_content: list = []
5          for block in message.content:
6              if isinstance(block, dict):
7                  block_type = block.get("type")
8                  if block_type == "text":
9                      # Strip annotations
10                     new_content.append({"type": "text", "text":
block["text"]})
11                 elif block_type in ("reasoning", "tool_call"):
12                     pass
13                 else:
14                     new_content.append(block)
15             else:
16                 new_content.append(block)
17             return message.model_copy(update={"content": new_content})
18
19     return message

```

上述代码的作用是将遵循 `v1规范` 的消息转换为 `Chat Completions格式`，这一过程不涉及 `reasoning_content` 字段的处理。

3. `_get_request_payload()`

源码位于 `langchain_openai.chat_models.base.BaseChatOpenAI` 的 `_get_request_payload()` 方法中，如下

```

1  def _get_request_payload(
2      self,
3      input_: LanguageModelInput,
4      *,
5      stop: list[str] | None = None,
6      **kwargs: Any,
7  ) -> dict:
8      messages = self._convert_input(input_).to_messages()
9      if stop is not None:
10         kwargs["stop"] = stop

```

```

11         payload = {**self._default_params, **kwargs}
12
13         if self._use_responses_api(payload):
14             if self.use_previous_response_id:
15                 last_messages, previous_response_id =
16                 _get_last_messages(messages)
17                 payload_to_use = last_messages if previous_response_id else
18                 messages
19                 if previous_response_id:
20                     payload["previous_response_id"] = previous_response_id
21                     payload = _construct_responses_api_payload(payload_to_use,
22                     payload)
23             else:
24                 payload = _construct_responses_api_payload(messages,
25                 payload)
26             else:
27                 payload["messages"] = [
28                     _convert_message_to_dict(_convert_from_v1_to_chat_completions(m))
29                     if isinstance(m, AIMessage)
30                     else _convert_message_to_dict(m)
31                     for m in messages
32                 ]
33             return payload

```

该方法调用 `_convert_message_to_dict()` 将消息列表转换为字典列表。

4. `_get_request_payload()`

源码位于 `langchain_deepseek.chat_models.ChatDeepSeek` 的 `_get_request_payload()` 方法中，如下

```

1     def _get_request_payload(
2         self,
3         input_: LanguageModelInput,
4         *,
5         stop: list[str] | None = None,
6         **kwargs: Any,
7     ) -> dict:
8         payload = super()._get_request_payload(input_, stop=stop, **kwargs)
9         for message in payload["messages"]:
10             if message["role"] == "tool" and isinstance(message["content"],
11             list):
12                 message["content"] = json.dumps(message["content"])
13             elif message["role"] == "assistant" and isinstance(
14                 message["content"], list
15             ):
16                 # DeepSeek API expects assistant content to be a string, not
17                 a list.
18                 # Extract text blocks and join them, or use empty string if
19                 none exist.
20                 text_parts = [
21                     block.get("text", "")
22                     for block in message["content"]
23                     if isinstance(block, dict) and block.get("type") ==
24                     "text"

```

```

21         ]
22         message["content"] = "".join(text_parts) if text_parts else
    """
23         return payload

```

该方法重写并调用父类的同名方法，在其基础上对消息做了一些额外处理，但不涉及 `reasoning_content` 的处理。

5. 总结

阅读源码可知，`Langchain` 将消息列表转换为请求体 `payload` 时，不会将 `reasoning_content` 添加到 `payload` 的 `messages` 字段中。思考内容被舍弃了。

② 调试

调试3. 问题复现一节的代码，拿到 `ChatDeepSeek` 的方法 `_get_request_payload()` 返回的 `payload`，后者是最终发送给深度求索模型服务的请求体，在第二轮模型调用时，`payload` 内容如下

```

1  {
2      "extra_body": {
3          "thinking": {
4              "type": "enabled"
5          }
6      },
7      "messages": [
8          {
9              "content": "今天北京天气如何？ ",
10             "role": "user"
11         },
12         {
13             "content": "None",
14             "role": "assistant",
15             "tool_calls": [
16                 {
17                     "function": {
18                         "arguments": "{\"city\": \"北京\"}",
19                         "name": "get_weather"
20                     },
21                     "id": "call_00_cYd22Kd9ApRsYZP45WDU7084",
22                     "type": "function"
23                 }
24             ]
25         },
26         {
27             "content": "北京今天天气不错",
28             "role": "tool",
29             "tool_call_id": "call_00_cYd22Kd9ApRsYZP45WDU7084"
30         }
31     ],
32     "model": "deepseek-v4-flash",
33     "stream": "False",
34     "tools": [
35         {
36             "function": {
37                 "description": "获取指定城市天气",
38                 "name": "get_weather",
39                 "parameters": {
40                     "properties": {

```

```
41         "city": {
42             "description": "城市名称",
43             "type": "string"
44         }
45     },
46     "required": [
47         "city"
48     ],
49     "type": "object"
50 },
51 },
52 "type": "function"
53 }
54 ]
55 }
```

可见，历史Assistant消息缺少 `reasoning_content` 字段，由上文分析可知，这样的请求体会触发报错信息

```
1 'message': 'The `reasoning_content` in the thinking mode must be passed back to the API.'
```

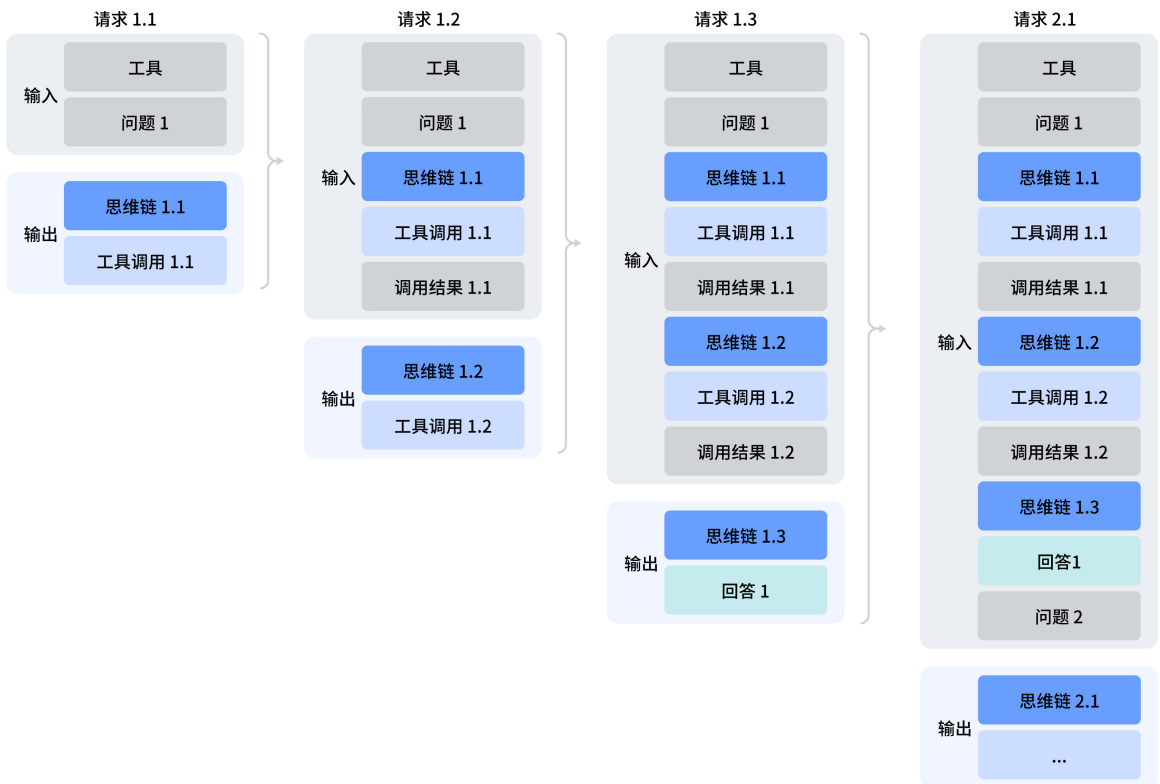
这就是问题触发的根本原因。

2.4 问题总结

2.4.1 背景知识

① 思考模式+工具调用下的消息列表格式要求

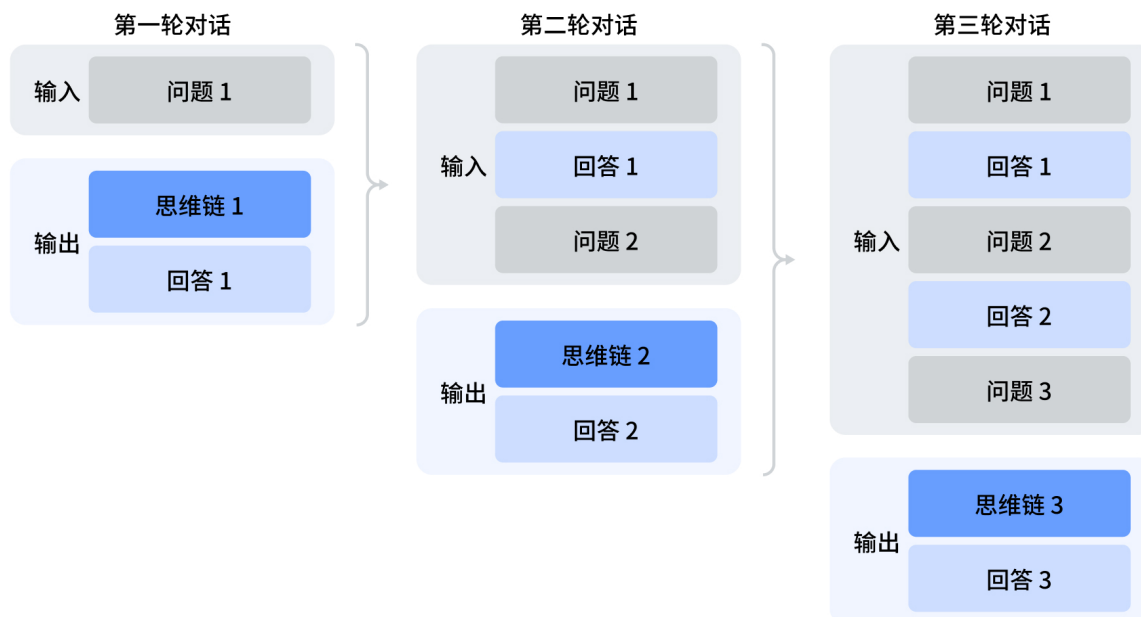
Deepseek 官方要求，在启用思考模式且调用工具时，必须将思考内容回传，如下图所示。



而根据官方的API规范，思考内容必须作为一级字段 `reasoning_content` 存储在消息对象中。

② 思考模式且无工具调用时的消息列表格式要求

启用思考模式，但没有工具调用请求时，根据[官方文档](#)，消息列表格式要求如下



此时并不需要回传思维链，和Langchain的行为一致，所以此时不会报错。

2.4.2 总结

0. **核心诱因**：问题触发的核心在于**思考过程+工具调用**，仅当启用模型的思考模式、且存在工具调用时，相关问题才会触发。
1. **存储机制**：通过 LangChain 调用 DeepSeek 模型时，生成的思考内容被持久化于 `AIMessage` 对象的 `additional_kwargs` 字段下，键名为 `reasoning_content`。
2. **状态丢失**：在多轮对话流中，LangChain 的标准序列化组件在发送API请求时，不会自动提取并回传历史消息中的思考内容，导致模型获取的**信息缺失**。
3. **非工具调用场景**：此场景下深度求索官方不要求回传思维链，Langchain的行为和官方要求一致，此时不会有**问题**。
4. **工具调用场景**：由于 DeepSeek API 协议要求在存在工具调用时，必须回传前序 `Assistant` 消息的完整状态（包含思考内容），缺失该字段将直接触发 **缺少 `reasoning_content`** 错误，导致请求失败。

2.4.3 验证

① 非工具调用多轮对话场景

```
1 from langchain_deepseek import ChatDeepSeek
2 from langchain.messages import HumanMessage
3 from dotenv import load_dotenv
4 load_dotenv(override=True)
5
6 model = ChatDeepSeek(
7     model="deepseek-reasoner"
8 )
9 messages = [HumanMessage("你好，我是老王")]
10
11 response1 = model.invoke(messages)
12 messages.append(response1)
13
14 messages.append(HumanMessage("你好，我是谁？"))
15 response2 = model.invoke(messages)
```

```
16 messages.append(response2)
17
18 for msg in messages:
19     print("=" * 80)
20     print(msg)
```

输出如下

```

1 =====
2 content='你好，我是老王' additional_kwargs={} response_metadata={}
3 =====
4 content='老王你好！我是DeepSeek，很高兴认识你。有什么我可以帮忙的吗？无论是闲聊、解答问题，还是讨论感兴趣的话题，我都乐意奉陪。你最近在忙些什么呢？' additional_kwargs=
{'refusal': None, 'reasoning_content': '嗯，用户说“你好，我是老王”，这是一个简单的自我介绍。没有提出具体问题或请求帮助。我需要友好地回应，并主动开启对话，为后续交流铺路。可以问候老王，提示我可以提供的帮助范围，比如知识解答、问题讨论、闲聊或者文件处理之类的。语气要热情、自然，称呼“老王”比较亲切。'} response_metadata={'token_usage':
{'completion_tokens': 115, 'prompt_tokens': 8, 'total_tokens': 123,
'completion_tokens_details': {'accepted_prediction_tokens': None,
'audio_tokens': None, 'reasoning_tokens': 74, 'rejected_prediction_tokens':
None}, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0},
'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 8},
'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash',
'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'id':
'816da21d-bfc5-4d0f-81ac-a737d214beac', 'finish_reason': 'stop', 'logprobs':
None} id='lc_run--019dfcfb-b0b2-7103-aca4-08a382509f36-0' tool_calls=[]
invalid_tool_calls=[] usage_metadata={'input_tokens': 8, 'output_tokens':
115, 'total_tokens': 123, 'input_token_details': {'cache_read': 0},
'output_token_details': {'reasoning': 74}}
5 =====
6 content='你好，我是谁？' additional_kwargs={} response_metadata={}
7 =====
8 content='哈，老王，你当然是你自己啊！不过根据咱们刚才的对话，你可是自称“老王”的——所以现在的
问题答案是：**你是老王**，一位正在和DeepSeek聊天的朋友。😄\n\n是突然想确认一下身份，还是
跟我开玩笑呢？' additional_kwargs={'refusal': None, 'reasoning_content': '好的，用户问“你好，我是谁？”。这个问题有点意思，刚刚他明明已经自我介绍说“我是老王”了，现在却问
“我是谁”。\n\n嗯，我得先理解他的真实意图。可能他在测试我是否记得刚才的对话？或者他是在开玩笑、想确认我有没有认真听他说话？也可能他一时忘记了自己刚才的自我介绍，或者想看看AI的上下文记忆能力。
\n\n考虑到对话历史里，他上次明确说“我是老王”，而且我回复时也用了“老王你好”。那么现在他问“我是谁”，最合理的解释是他在看我有没有记住他刚才的自我介绍。深层需求可能是想确认AI的对话
连贯性和记忆力，或者是一种社交性的互动，想让我用更亲切的方式再次确认他的身份。
\n\n所以，我应该基于已有的对话历史，肯定地告诉他“你是老王”，并且可以带点幽默感，比如加个括号注明“根据刚才的自我介绍”，这样既确认了信息，又显得自然亲切。毕竟他主动说过，我就应该记住并回应这个身份信息。'}
response_metadata={'token_usage': {'completion_tokens': 276,
'prompt_tokens': 57, 'total_tokens': 333, 'completion_tokens_details':
{'accepted_prediction_tokens': None, 'audio_tokens': None,
'reasoning_tokens': 215, 'rejected_prediction_tokens': None},
'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0},
'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 57},
'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash',
'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'id':
'dd21f28c-1075-4e2d-8bd3-c225c7342fcc', 'finish_reason': 'stop', 'logprobs':
None} id='lc_run--019dfcfb-b9f8-7470-a7a8-4e57fcaba94c-0' tool_calls=[]
invalid_tool_calls=[] usage_metadata={'input_tokens': 57, 'output_tokens':
276, 'total_tokens': 333, 'input_token_details': {'cache_read': 0},
'output_token_details': {'reasoning': 215}}

```

此时不会报错。

② 工具调用多轮对话场景

抛异常，见上文3. 问题复现。

3、解决方案

3.1 规避问题

既然问题是在**工具调用+思考模式**触发的，那么只要打破这个组合，就可以规避问题。工具调用是为了拓展模型能力，自然不能禁用，解决方案就只有在工具调用时禁用思考模式了。

3.1.1 通过参数禁用思考模式

只需要将 **thinking** 字段下的 **enabled** 替换为 **disabled** 即可。

```
1 from langchain_deepseek import ChatDeepSeek
2 from langchain.messages import HumanMessage
3 from langchain.tools import tool
4 from dotenv import load_dotenv
5 load_dotenv(override=True)
6
7 @tool(parse_docstring=True)
8 def get_weather(city: str):
9     """
10     获取指定城市天气
11
12     Args:
13         city: 城市名称
14     """
15     return f"{city}今天天气不错"
16
17 model = ChatDeepSeek(
18     model="deepseek-v4-flash",
19     extra_body={
20         "thinking": {
21             "type": "disabled"
22         }
23     }
24 )
25 model_with_tools = model.bind_tools([get_weather])
26 messages = [HumanMessage("今天北京天气如何? ")]
27
28 response1 = model_with_tools.invoke(messages)
29 messages.append(response1)
30
31 for tool_call in response1.tool_calls:
32     if tool_call["name"] == "get_weather":
33         tool_msg = get_weather.invoke(tool_call)
34         messages.append(tool_msg)
35
36 response = model_with_tools.invoke(messages)
37 messages.append(response)
38
39 for msg in messages:
40     print("=" * 80)
41     print(msg)
```

输出如下

```
1 =====
2 content='今天北京天气如何?' additional_kwargs={} response_metadata={}
3 =====
4 content='好的，我来查询北京今天的天气情况。' additional_kwargs={'refusal': None}
  response_metadata={'token_usage': {'completion_tokens': 53, 'prompt_tokens':
  283, 'total_tokens': 336, 'completion_tokens_details': None,
  'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 256},
  'prompt_cache_hit_tokens': 256, 'prompt_cache_miss_tokens': 27},
  'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash',
  'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'id':
  '65c88d9f-7ad8-466c-a501-369d270ed8d1', 'finish_reason': 'tool_calls',
  'logprobs': None} id='lc_run--019e0118-8b00-7542-8336-febfe570237b-0'
  tool_calls=[{'name': 'get_weather', 'args': {'city': '北京'}, 'id':
  'call_00_HnmKmfCWyJy2aIsFtNf48838', 'type': 'tool_call'}] invalid_tool_calls=
  [] usage_metadata={'input_tokens': 283, 'output_tokens': 53, 'total_tokens':
  336, 'input_token_details': {'cache_read': 256}, 'output_token_details': {}}
5 =====
6 content='北京今天天气不错' name='get_weather'
  tool_call_id='call_00_HnmKmfCWyJy2aIsFtNf48838'
7 =====
8 content='根据查询结果，今天北京的天气**不错**，是个适合外出活动的好天气！☁️\n\n如果您需要
  更详细的信息，比如温度、风力、湿度等，我这边暂时无法提供更具体的数据。建议您打开天气App
  或网站查看更详细的天气预报。' additional_kwargs={'refusal': None}
  response_metadata={'token_usage': {'completion_tokens': 57, 'prompt_tokens':
  351, 'total_tokens': 408, 'completion_tokens_details': None,
  'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 256},
  'prompt_cache_hit_tokens': 256, 'prompt_cache_miss_tokens': 95},
  'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash',
  'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'id':
  '0606945c-a6a2-4531-ac55-d62c5ba001e4', 'finish_reason': 'stop', 'logprobs':
  None} id='lc_run--019e0118-9013-74b3-9a38-78333dbfdf5e-0' tool_calls=[]
  invalid_tool_calls=[] usage_metadata={'input_tokens': 351, 'output_tokens':
  57, 'total_tokens': 408, 'input_token_details': {'cache_read': 256},
  'output_token_details': {}}
```

3.1.2 直接调用非思考模型

参考官方文档的[模型细节](#)，如下

快速开始		模型版本	DeepSeek-V4-Flash	DeepSeek-V4-Pro
首次调用 API		思考模式	支持非思考与思考模式 (默认) 切换方式详见思考模式	
模型 & 价格		上下文长度	1M	
Token 用量计算		输出长度	最大 384K	
限速		Json Output	支持	支持
错误码		Tool Calls	支持	支持
接入 Agent 工具		对话前缀续写 (Beta)	支持	支持
API 指南		FIM 补全 (Beta)	仅非思考模式支持	仅非思考模式支持
思考模式		百万tokens输入 (缓存命中) ⁽²⁾	0.02元	0.025元 (2.5折 ⁽³⁾) 0.1元
多轮对话		百万tokens输入 (缓存未命中)	1元	3元 (2.5折 ⁽³⁾) 12元
对话前缀续写 (Beta)		百万tokens输出	2元	6元 (2.5折 ⁽³⁾) 24元
FIM 补全 (Beta)		(1) deepseek-chat 与 deepseek-reasoner 两个模型名将于日后弃用。出于兼容考虑，二者分别对应 deepseek-v4-flash 的非思考与思考模式。		
JSON Output		(2) 全系列模型，输入缓存命中的价格已降至首发价格的 1/10，该价格调整自北京时间 2026/4/26 20:15 起生效		
Tool Calls		(3) 当前 deepseek-v4-pro 模型 2.5 折，优惠期延长至北京时间 2026/05/31 23:59。		
上下文硬盘缓存		扣费规则		
Anthropic API				
API 文档				
基本信息				
对话 (Chat)				
对话补全				
补全 (Completions)				

deepseek-chat 和 **deepseek-reasoner** 分别路由至 **deepseek-v4-flash** 的非思考与思考模式。

因此，可以直接将模型ID替换为 **deepseek-chat**，如下

```
1 from langchain_deepseek import ChatDeepSeek
2 from langchain.messages import HumanMessage
3 from langchain.tools import tool
4 from dotenv import load_dotenv
5 load_dotenv(override=True)
6
7 @tool(parse_docstring=True)
8 def get_weather(city: str):
9     """
10     获取指定城市天气
11
12     Args:
13         city: 城市名称
14     """
15     return f"{city}今天天气不错"
16
17 model = ChatDeepSeek(
18     model="deepseek-chat"
19 )
20 model_with_tools = model.bind_tools([get_weather])
21 messages = [HumanMessage("今天北京天气如何?")]
22
23 response1 = model_with_tools.invoke(messages)
24 messages.append(response1)
25
26 for tool_call in response1.tool_calls:
27     if tool_call["name"] == "get_weather":
28         tool_msg = get_weather.invoke(tool_call)
29         messages.append(tool_msg)
30
31 response = model_with_tools.invoke(messages)
32 messages.append(response)
33
34 for msg in messages:
35     print("=" * 80)
```

输出如下

```

1  =====
   ===
2  content='今天北京天气如何?' additional_kwargs={} response_metadata={}
3  =====
   ===
4  content='好的，我来查一下今天北京的天气情况。' additional_kwargs={'refusal': None}
   response_metadata={'token_usage': {'completion_tokens': 54, 'prompt_tokens':
   283, 'total_tokens': 337, 'completion_tokens_details': None,
   'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 256},
   'prompt_cache_hit_tokens': 256, 'prompt_cache_miss_tokens': 27},
   'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash',
   'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'id':
   '11fda3b7-a7ea-4944-acf9-06420b414c43', 'finish_reason': 'tool_calls',
   'logprobs': None} id='lc_run--019e011d-9c79-7601-b890-225031789b3f-0'
   tool_calls=[{'name': 'get_weather', 'args': {'city': '北京'}, 'id':
   'call_00_d2pNbOLAC6PILUjM1hy31171', 'type': 'tool_call'}] invalid_tool_calls=
   [] usage_metadata={'input_tokens': 283, 'output_tokens': 54, 'total_tokens':
   337, 'input_token_details': {'cache_read': 256}, 'output_token_details': {}}
5  =====
   ===
6  content='北京今天天气不错' name='get_weather'
   tool_call_id='call_00_d2pNbOLAC6PILUjM1hy31171'
7  =====
   ===
8  content='今天北京的天气不错，是个适合出门的好日子！☺️\n\n不过，如果您需要更详细的信息（如
   具体温度、风力、湿度等），建议您使用专业的天气应用或网站查询，这样能获得更准确的实时数据。有
   什么其他需要帮忙的吗?' additional_kwargs={'refusal': None} response_metadata=
   {'token_usage': {'completion_tokens': 55, 'prompt_tokens': 352,
   'total_tokens': 407, 'completion_tokens_details': None,
   'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 256},
   'prompt_cache_hit_tokens': 256, 'prompt_cache_miss_tokens': 96},
   'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash',
   'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'id':
   'd3621ed5-2af8-4fba-9752-229e419a9ffd', 'finish_reason': 'stop', 'logprobs':
   None} id='lc_run--019e011d-a13b-7813-aba4-72841fb1a84d-0' tool_calls=[]
   invalid_tool_calls=[] usage_metadata={'input_tokens': 352, 'output_tokens':
   55, 'total_tokens': 407, 'input_token_details': {'cache_read': 256},
   'output_token_details': {}}

```

但要注意，这两个ID只是处于兼容考虑短暂保留，将于 **deepseek-v4** 发布之日起三月后弃用，参考 [Deepseek官方公众号关于V4预览版发布的文章](#)，如下

V4-Pro 与 V4-Flash 最大上下文长度为 1M, 均同时支持**非思考模式**与**思考模式**, 其中思考模式支持 `reasoning_effort` 参数设置思考强度 (high/max)。对于复杂的 Agent 场景建议使用思考模式, 并设置强度为 max。模型调用与参数调整方法请参考 API 文档:

https://api-docs.deepseek.com/zh-cn/guides/thinking_mode

请大家注意: 旧有的 API 接口的两个模型名 `deepseek-chat` 与 `deepseek-reasoner` 将于三个月后 (2026-07-24) 停止使用。当前阶段内, 这两个模型名分别指向 `deepseek-v4-flash` 的**非思考模式**与**思考模式**。

开源权重和本地部署

- DeepSeek-V4 模型开源链接:

<https://huggingface.co/collections/deepseek-ai/deepseek-v4>

<https://modelscope.cn/collections/deepseek-ai/DeepSeek-V4>

- DeepSeek-V4 技术报告:

这种方法会在不久后失效。

3.2 补齐信息

最稳妥的办法是将 `reasoning_content` 添加到 `payload` 的 `messages` 字段中

此处定义 `ChatDeepSeek` 子类, 重写 `_get_request_payload()`

```
1 from langchain_deepseek import ChatDeepSeek
2 from langchain.messages import HumanMessage
3 from langchain.tools import tool
4
5 from dotenv import load_dotenv
6 load_dotenv(override=True)
7
8 class ChatDeepSeekReasoning(ChatDeepSeek):
9     def _get_request_payload(
10         self,
11         input_,
12         *,
13         stop: list[str] | None = None,
14         **kwargs,
15     ) -> dict:
16         # 1. 获取父类生成的标准 OpenAI 格式 payload
17         payload = super()._get_request_payload(input_, stop=stop, **kwargs)
18
19         # 2. 安全地回传 reasoning_content
20         messages_in_payload = payload.get("messages", [])
21
22         # 确保 input_ (BaseMessage列表) 和 payload 里的 dict 列表长度一致
23         if len(input_) != len(messages_in_payload):
24             # 如果长度不一致, 说明框架内部做了特殊处理 (如合并或过滤)
25             # 此时建议通过更复杂的逻辑匹配, 或者记录日志
26             logger.warning("input_ 和 payload 的 messages 长度不一致")
27             return payload
28
29         for idx, msg in enumerate(input_):
30             # 获取原消息中的思考内容 (从 dict 中拿)
31             reasoning = msg.additional_kwargs.get("reasoning_content")
```

```

32
33         # 只有当它是 AIMessage 且确实有内容，且 payload 里还没填入时才注入
34         if isinstance(msg, AIMessage) and reasoning:
35             target_dict = messages_in_payload[idx]
36
37         # 检查 payload 里的这个字典，确保它没有 reasoning_content 字段
38         if "reasoning_content" not in target_dict:
39             target_dict["reasoning_content"] = reasoning
40
41         return payload
42
43     @tool(parse_docstring=True)
44     def get_weather(city: str):
45         """
46         获取指定城市天气
47
48         Args:
49             city: 城市名称
50         """
51         return f"{city}今天天气不错"
52
53     model = ChatDeepSeekReasoning(
54         model="deepseek-v4-flash",
55         extra_body={
56             "thinking": {
57                 "type": "enabled"
58             }
59         }
60     )
61     model_with_tools = model.bind_tools([get_weather])
62     messages = [HumanMessage("今天北京天气如何? ")]
63
64     response1 = model_with_tools.invoke(messages)
65     messages.append(response1)
66
67     for tool_call in response1.tool_calls:
68         if tool_call["name"] == "get_weather":
69             tool_msg = get_weather.invoke(tool_call)
70             messages.append(tool_msg)
71
72     response = model_with_tools.invoke(messages)
73     messages.append(response)
74
75     for msg in messages:
76         print("=" * 80)
77         print(msg)

```

输出如下

```

1 =====
2 content='今天北京天气如何?' additional_kwargs={} response_metadata={}
3 =====
4 content='' additional_kwargs={'refusal': None, 'reasoning_content': '用户想知道
北京的天气情况。我可以使用 get_weather 工具来获取北京今天的天气信息。'}
response_metadata={'token_usage': {'completion_tokens': 65, 'prompt_tokens':
283, 'total_tokens': 348, 'completion_tokens_details':
{'accepted_prediction_tokens': None, 'audio_tokens': None,
'reasoning_tokens': 20, 'rejected_prediction_tokens': None},
'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 256},
'prompt_cache_hit_tokens': 256, 'prompt_cache_miss_tokens': 27},
'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash',
'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'id':
'2180b539-ad6f-4869-8edc-4575d972d0a7', 'finish_reason': 'tool_calls',
'logprobs': None} id='lc_run--019e0127-b3bc-7c40-97fb-147e2eab578f-0'
tool_calls=[{'name': 'get_weather', 'args': {'city': '北京'}, 'id':
'call_00_YIN8toAGBbnU5bGqT7B41615', 'type': 'tool_call'}] invalid_tool_calls=
[] usage_metadata={'input_tokens': 283, 'output_tokens': 65, 'total_tokens':
348, 'input_token_details': {'cache_read': 256}, 'output_token_details':
{'reasoning': 20}}
5 =====
6 content='北京今天天气不错' name='get_weather'
tool_call_id='call_00_YIN8toAGBbnU5bGqT7B41615'
7 =====
8 content='今天北京的天气不错哦! ☺ 是个适合出门的好天气。如果你有出行计划, 可以放心安排~ 不
过如果需要更具体的温度、风力等信息, 也可以随时问我! 😊' additional_kwargs={'refusal':
None, 'reasoning_content': '工具返回了"北京今天天气不错", 这是一个简单的回答。我就把这
个信息告诉用户。'} response_metadata={'token_usage': {'completion_tokens': 61,
'prompt_tokens': 364, 'total_tokens': 425, 'completion_tokens_details':
{'accepted_prediction_tokens': None, 'audio_tokens': None,
'reasoning_tokens': 19, 'rejected_prediction_tokens': None},
'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 256},
'prompt_cache_hit_tokens': 256, 'prompt_cache_miss_tokens': 108},
'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash',
'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'id':
'a846c148-707c-481e-9a8e-568fd12f55c0', 'finish_reason': 'stop', 'logprobs':
None} id='lc_run--019e0127-ba29-7ac2-b196-f6b887f1a2e8-0' tool_calls=[]
invalid_tool_calls=[] usage_metadata={'input_tokens': 364, 'output_tokens':
61, 'total_tokens': 425, 'input_token_details': {'cache_read': 256},
'output_token_details': {'reasoning': 19}}

```

3.3 特殊解法

3.3.1 解决方案

实测发现, 将模型ID设置为 `deepseek-reasoner` 时调用工具也不会报错。

根据官方文档, `deepseek-reasoner` 等价于 `deepseek-v4-flash` 的思考模式, 实测与官方说明一致。并且调试发现, `payload` 也不包含 `reasoning_content`。

根据上文分析, 这种情况应该触发报错, 但实测可以运行, 如下。

```
1 from langchain_deepseek import ChatDeepSeek
```

```

2  from langchain.messages import HumanMessage
3  from langchain.tools import tool
4  from dotenv import load_dotenv
5  load_dotenv(override=True)
6
7  @tool(parse_docstring=True)
8  def get_weather(city: str):
9      """
10     获取指定城市天气
11
12     Args:
13         city: 城市名称
14     """
15     return f"{city}今天天气不错"
16
17  model = ChatDeepSeek(
18     model="deepseek-reasoner"
19 )
20  model_with_tools = model.bind_tools([get_weather])
21  messages = [HumanMessage("今天北京天气如何? ")]
22
23  response1 = model_with_tools.invoke(messages)
24  messages.append(response1)
25
26  for tool_call in response1.tool_calls:
27      if tool_call["name"] == "get_weather":
28          tool_msg = get_weather.invoke(tool_call)
29          messages.append(tool_msg)
30
31  response = model_with_tools.invoke(messages)
32  messages.append(response)
33
34  for msg in messages:
35      print("=" * 80)
36      print(msg)

```

输出如下


```

1 =====
2 content='今天北京天气如何?' additional_kwargs={} response_metadata={}
3 =====
4 content='' additional_kwargs={'refusal': None, 'reasoning_content': '用户想知道
北京今天的天气。我可以使用get_weather工具来获取北京天气信息。'} response_metadata=
{'token_usage': {'completion_tokens': 63, 'prompt_tokens': 283,
'total_tokens': 346, 'completion_tokens_details':
{'accepted_prediction_tokens': None, 'audio_tokens': None,
'reasoning_tokens': 18, 'rejected_prediction_tokens': None},
'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 256},
'prompt_cache_hit_tokens': 256, 'prompt_cache_miss_tokens': 27},
'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash',
'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'id':
'38ccce43-630e-4ce7-974a-26b8b680a2bf', 'finish_reason': 'tool_calls',
'logprobs': None} id='lc_run--019e0130-b8ec-7621-b50b-aff07e481507-0'
tool_calls=[{'name': 'get_weather', 'args': {'city': '北京'}, 'id':
'call_00_AHJwti524A6FDW6ppmi18815', 'type': 'tool_call'}] invalid_tool_calls=
[] usage_metadata={'input_tokens': 283, 'output_tokens': 63, 'total_tokens':
346, 'input_token_details': {'cache_read': 256}, 'output_token_details':
{'reasoning': 18}}
5 =====
6 content='北京今天天气不错' name='get_weather'
tool_call_id='call_00_AHJwti524A6FDW6ppmi18815'
7 =====
8 content='今天北京的天气不错哦! ☺ 是个适合出门的好天气~' additional_kwargs=
{'refusal': None, 'reasoning_content': 'The user asked about today\'s weather
in Beijing, and the tool returned "北京今天天气不错" which means "Beijing\'s
weather today is nice/good."'} response_metadata={'token_usage':
{'completion_tokens': 49, 'prompt_tokens': 344, 'total_tokens': 393,
'completion_tokens_details': {'accepted_prediction_tokens': None,
'audio_tokens': None, 'reasoning_tokens': 32, 'rejected_prediction_tokens':
None}, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 256},
'prompt_cache_hit_tokens': 256, 'prompt_cache_miss_tokens': 88},
'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash',
'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'id':
'52969ad5-1cb2-4a76-a4e8-bb3f7b966205', 'finish_reason': 'stop', 'logprobs':
None} id='lc_run--019e0130-bfa8-7103-9844-229666c776b2-0' tool_calls=[]
invalid_tool_calls=[] usage_metadata={'input_tokens': 344, 'output_tokens':
49, 'total_tokens': 393, 'input_token_details': {'cache_read': 256},
'output_token_details': {'reasoning': 32}}

```

3.3.2 原因分析

为了确定真实原因，绕过Langchain直接发送请求，命令如下

```

1 curl -L -X POST 'https://api.deepseek.com/chat/completions' \
2 -H 'Content-Type: application/json' \
3 -H 'Accept: application/json' \
4 -H 'Authorization: Bearer <TOKEN>' \
5 --data-raw '{
6   "messages": [
7     {
8       "content": "今天北京天气如何?",

```

```

9      "role": "user"
10    },
11    {
12      "content": "None",
13      "role": "assistant",
14      "tool_calls": [
15        {
16          "function": {
17            "arguments": "{\"city\": \"北京\"}",
18            "name": "get_weather"
19          },
20          "id": "call_00_e82qQYXWR9mYwcdwNCDo1725",
21          "type": "function"
22        }
23      ]
24    },
25    {
26      "content": "北京今天天气不错",
27      "role": "tool",
28      "tool_call_id": "call_00_e82qQYXWR9mYwcdwNCDo1725"
29    }
30  ],
31  "model": "deepseek-reasoner"
32 }'

```

输出如下

```

1  {
2    "id": "e279f10f-d5bf-4114-b259-2b2e8b7b08c9",
3    "object": "chat.completion",
4    "created": 1778136528,
5    "model": "deepseek-v4-flash",
6    "choices": [
7      {
8        "index": 0,
9        "message": {
10          "role": "assistant",
11          "content": "北京今天天气不错，适合外出活动。\\n\\n根据目前的信息，北京今天以晴天或多云为主，气温大约在 **-3℃ 到 9℃** 之间，白天体感比较舒适，但早晚温差较大，风力不大，空气质量也比较好。\\n\\n**温馨提示:**\\n- 建议穿厚外套、羽绒服，注意早晚保暖。\\n- 紫外线中等，如果长时间户外活动可以适当防晒。\\n\\n如果你想了解更精确的实时气温或未来几天的详细预报，建议打开联网搜索功能查询最新数据。",
12          "reasoning_content": "好的，用户问的是“今天北京天气如何”，这是一个需要实时信息的问题。我刚刚通过调用工具获取了北京的天气信息，返回的结果是“北京今天天气不错”。这个描述比较概括，没有具体数据。不过，基于我已有的知识，我可以提供一些更常见的北京天气特征作为补充，比如气温范围、昼夜温差、是否有风或降雨趋势等。这样回答会更完整，对用户更有帮助。"
13        },
14        "logprobs": null,
15        "finish_reason": "stop"
16      }
17    ],
18    "usage": {
19      "prompt_tokens": 71,
20      "completion_tokens": 198,
21      "total_tokens": 269,
22      "prompt_tokens_details": {

```

```
23         "cached_tokens": 0
24     },
25     "completion_tokens_details": {
26         "reasoning_tokens": 88
27     },
28     "prompt_cache_hit_tokens": 0,
29     "prompt_cache_miss_tokens": 71
30 },
31 "system_fingerprint": "fp_058df29938_prod0820_fp8_kvcache_20260402"
32 }
```

可以确定，上述现象出现的原因和Langchain无关，是深度求索官方对于 **deepseek-reasoner** 的处理更加宽松，允许在工具调用且启用思考模式时丢失 **reasoning_content**。

但注意：

1. **deepseek-reasoner** 将于 **2026-07-24** 弃用，上述解法将失效。
2. 虽然官方对于 **deepseek-reasoner** 的处理更加宽松，但丢失思考内容，不符合官方文档的要求，不建议使用。